Lecture 27: Practice Problems Suite (Questions)

CreditWise Loan Approval System (Minor Project)

This problem set tests algorithmic foundations, data leakage mitigation, non-linear financial feature engineering, and cost-utility threshold optimization.

Problem 01: Automated Credit Data Auditor & Statistical Imputer (CreditRiskDataAuditor)

🎯 Objective & Requirements

Write a production-grade, scikit-learn compatible estimator or pipeline component solving this financial modeling challenge.

Mathematical Invariants:

1. All missing data imputations and transformations must strictly obey the `fit()` (training only) vs `transform()` (test/inference) separation.
2. Ensure mathematical operations are numerically stable ($\epsilon = 10^-6$ or $+1.0$ for ratios).
3. Code must assert clean input-output matrix shapes and types.

Problem 02: Robust Financial Categorical Feature Encoder (FinancialFeatureEncoder)

🎯 Objective & Requirements

Write a production-grade, scikit-learn compatible estimator or pipeline component solving this financial modeling challenge.

Mathematical Invariants:

1. All missing data imputations and transformations must strictly obey the `fit()` (training only) vs `transform()` (test/inference) separation.
2. Ensure mathematical operations are numerically stable ($\epsilon = 10^-6$ or $+1.0$ for ratios).
3. Code must assert clean input-output matrix shapes and types.

Problem 03: Tukey IQR Financial Outlier Capping Transformer (OutlierCappingTransformer)

🎯 Objective & Requirements

Write a production-grade, scikit-learn compatible estimator or pipeline component solving this financial modeling challenge.

Mathematical Invariants:

1. All missing data imputations and transformations must strictly obey the `fit()` (training only) vs `transform()` (test/inference) separation.
2. Ensure mathematical operations are numerically stable ($\epsilon = 10^-6$ or $+1.0$ for ratios).
3. Code must assert clean input-output matrix shapes and types.

Problem 04: Domain-Driven Financial Risk Ratio Generator (FinancialRatioGenerator)

🎯 Objective & Requirements

Write a production-grade, scikit-learn compatible estimator or pipeline component solving this financial modeling challenge.

Mathematical Invariants:

1. All missing data imputations and transformations must strictly obey the `fit()` (training only) vs `transform()` (test/inference) separation.
2. Ensure mathematical operations are numerically stable ($\epsilon = 10^-6$ or $+1.0$ for ratios).
3. Code must assert clean input-output matrix shapes and types.

Problem 05: Cost-Sensitive Logistic Regression Underwriter (CostSensitiveLogisticRegression)

🎯 Objective & Requirements

Write a production-grade, scikit-learn compatible estimator or pipeline component solving this financial modeling challenge.

Mathematical Invariants:

1. All missing data imputations and transformations must strictly obey the `fit()` (training only) vs `transform()` (test/inference) separation.
2. Ensure mathematical operations are numerically stable ($\epsilon = 10^-6$ or $+1.0$ for ratios).
3. Code must assert clean input-output matrix shapes and types.

Problem 06: Distance-Weighted KNN Underwriting Engine (KNNUnderwritingEngine)

🎯 Objective & Requirements

Write a production-grade, scikit-learn compatible estimator or pipeline component solving this financial modeling challenge.

Mathematical Invariants:

1. All missing data imputations and transformations must strictly obey the `fit()` (training only) vs `transform()` (test/inference) separation.
2. Ensure mathematical operations are numerically stable ($\epsilon = 10^-6$ or $+1.0$ for ratios).
3. Code must assert clean input-output matrix shapes and types.

Problem 07: Generative Gaussian Naive Bayes Risk Assessor (GaussianNBRiskAssessor)

🎯 Objective & Requirements

Write a production-grade, scikit-learn compatible estimator or pipeline component solving this financial modeling challenge.

Mathematical Invariants:

1. All missing data imputations and transformations must strictly obey the `fit()` (training only) vs `transform()` (test/inference) separation.
2. Ensure mathematical operations are numerically stable ($\epsilon = 10^-6$ or $+1.0$ for ratios).
3. Code must assert clean input-output matrix shapes and types.

Problem 08: Profit-Maximizing Underwriting Threshold Tuner (OptimalDecisionThresholdTuner)

🎯 Objective & Requirements

Write a production-grade, scikit-learn compatible estimator or pipeline component solving this financial modeling challenge.

Mathematical Invariants:

1. All missing data imputations and transformations must strictly obey the `fit()` (training only) vs `transform()` (test/inference) separation.
2. Ensure mathematical operations are numerically stable ($\epsilon = 10^{-}6$ or $+1.0$ for ratios).
3. Code must assert clean input-output matrix shapes and types.
